The Supply Chain Control Tower: A Plain-English Project Report

This report explains what this project is, why it matters, and how it works — written for anyone, technical or not. No prior knowledge of software, data science, or supply chain management is assumed.

FIGURE PLACEHOLDER

The Supply Chain Control Tower Dashboard

`./assets/dashboard_overview.png`

Table of Contents

The Corporate Problem

Most companies that manufacture and sell physical products still plan how much to make using a simple, old-fashioned rule: "when stock drops below a certain amount, order more." This sounds sensible, but it quietly costs businesses real money in three specific ways.

- The rule never looks ahead. It only reacts once the shelf is already empty, because it has no way of noticing a busy period coming until it has already arrived.
- The rule doesn't know how much a factory can actually produce in a day. A factory might only be able to make 150 units a day no matter how many units the rule says to order —

demand doesn't wait for the factory to catch up.

- Running out of stock isn't just a missed sale. Many business contracts include a financial penalty for failing to deliver on time — in this project, a real $100 penalty for every single unit that doesn't arrive when promised. This kind of delivery promise is usually called an SLA, short for "Service Level Agreement" — a fancy term for a contractual promise to deliver on time.

When customer demand outpaces what a factory can physically produce, and nobody saw it coming, a business doesn't just lose a sale — it gets handed a penalty invoice on top of everything else. This project exists to replace that guesswork with a system that sees the rush coming and calculates, in exact dollars, the cheapest way to avoid the penalty.

The Data Engine (Dataset)

Most planning tools still work from static spreadsheets — a snapshot of last month's or last quarter's sales, exported by hand and reviewed only occasionally. By the time anyone looks at it, the numbers are already out of date.

This project replaces that with a live, always-running simulator that behaves like an actual chain of store cash registers:

- It creates a brand-new, realistic sale every fraction of a second — a specific product, a quantity, a store location, and an exact time.
- It uses a data-generation tool called Faker, which is simply a piece of software that makes up realistic-looking information (like believable store addresses) so the simulated sales feel like a real, busy retail chain rather than repeated, robotic test data.
- Because it never stops, every other part of the system always has fresh information to react to — there is no "waiting for next week's export."

FIGURE PLACEHOLDER

Live POS Feed: Simulated Sales Streaming In

`./assets/pos_simulator_terminal.png`

This live feed is the foundation everything else depends on. A business can only prepare for a demand spike ahead of time if it can actually see that spike happening as it unfolds, not three weeks later in a spreadsheet.

The Technology (Under the Hood)

Five pieces of technology work together to make this system run. None of them need to be understood in technical detail — here is what each one does, in plain terms.

- **Redpanda** acts as the system's live nervous system. Just as nerves carry signals instantly between different parts of a body, Redpanda carries every sale, every forecast, and every plan between the different parts of this system the instant they happen, so nothing has to wait around for the next scheduled check-in.
- **Prophet** acts as the system's weather forecaster. It looks at recent sales patterns for each product and predicts what demand is likely to look like over the coming week, the same way a weather forecaster studies recent conditions to predict tomorrow.
- **Google OR-Tools** acts as the system's math engine for making financial trade-offs. Given a demand forecast, a factory's daily production limit, and the real dollar costs involved, it works out the single cheapest possible production plan — instantly weighing "make more now" against "hold extra stock" against "risk a penalty," the way a very fast, very careful accountant would.
- **DuckDB** acts as the system's filing cabinet. Every time a new plan is calculated, it gets filed away here — a small, fast, self-contained record book that always holds the latest plan for each product, ready to be looked up instantly.
- **Streamlit** acts as the system's window to the outside world. It turns everything happening behind the scenes into a live webpage — the actual screen a business person would look at — showing the numbers, the risk, and the plan in a format anyone can read at a glance.

The Implementation

Here is exactly what happens, step by step, from the moment one simulated sale occurs to the moment the dashboard reflects it.

- A simulated cash register generates a sale and sends it out as a small message — think of it as a digital receipt.
- That message travels instantly through the live nervous system (Redpanda), which sits between every part of the pipeline so that no program ever has to talk directly to another.

- The weather-forecaster service (Prophet) is always listening. It continuously builds up a running history of recent sales for each product, and periodically — once enough new sales have come in — predicts how much of each product will be needed over the next 7 days.
- That 7-day prediction is itself sent as a message to the math engine (OR-Tools).
- The math engine calculates the single cheapest possible production plan for the next 7 days — how much to manufacture each day — while respecting the factory's daily production limit and the warehouse's storage limit.
- That finished plan is filed away in the filing cabinet (DuckDB).
- The live dashboard (Streamlit) checks the filing cabinet every 3 seconds and displays the current plan: how much to produce each day, how much inventory that leaves on hand, the total cost, and — most importantly — a clear warning if any customer demand is at risk of going unmet.

No person touches any step of this process. A single simulated sale can flow all the way through forecasting and planning and appear as an updated plan on the dashboard within seconds.

The Outcome & Business Insights

The most valuable thing this system demonstrates is that it looks ahead and prepares, rather than only reacting to what has already happened.

FIGURE PLACEHOLDER

Dashboard Chart: Inventory Climbing Ahead of a Demand Spike

`./assets/dashboard_prebuild_chart.png`

This is best explained with a real, verified example pulled directly from this system's own test results. Imagine a factory that can produce at most 150 units a day, starting with zero units in the warehouse. Demand is barely anything on day one — just 10 units — but spikes to 300 units on day two.

- A traditional, reactive system only ever looks at today. On day one it makes exactly 10 units. On day two, it can only make 150 more units, no matter how high demand climbs. That leaves 150 units of customer demand completely unmet.
- This system looks at both days at once. It recognizes that producing the full 150 units on day one — even though day one only needs 10 — lets it store the extra 140 units overnight, ready for day two's rush.

- Here is the verified mathematical floor: when a 300-unit demand spike hits a 150-unit daily production cap with zero starting inventory, a 10-unit residual stockout is mathematically unavoidable, no matter how the production is planned. There simply isn't enough total factory capacity across the two days to make up the entire difference. This system's math engine successfully finds that exact floor — the smallest possible shortfall — rather than falling short by a much larger, avoidable amount.
- Storing those 140 extra units overnight costs a small fee — $1.50 per unit — while every unit of unmet demand costs $100. Once the numbers are laid out this way, "hold a little extra stock ahead of a known rush" stops being a guess and becomes an obvious, provable financial decision.

This calculation happens automatically, for every product, every time new demand information arrives — with no person needing to notice the spike coming themselves.

Tech Defense

Building this system surfaced several real technical problems, each one caught and solved along the way rather than papered over.

- Early on, the forecasting tool (Prophet) was asked to predict 7 days ahead using data measured in minutes rather than days, because the demo compresses time to stay fast. Left unfixed, this produced a forecast that exploded to over 30,000 units from input data that never rose above 25 — a textbook case of a model doing exactly what it was told, on the wrong scale. It was fixed by switching the forecasting mode to one that trusts the recent average rather than an extrapolated trend, which is the statistically honest choice with only a small amount of data.
- The math engine (OR-Tools) originally assumed a factory could produce an unlimited amount per day, so it could never actually fail. Once a real 150-unit daily cap was introduced, a large enough demand spike would have made the model return "no solution exists" at the exact moment a plan was needed most. This was solved by adding a specific new variable representing unmet demand, priced at the real $100 penalty, so the system always produces a valid plan and always states, in dollars, exactly what a shortfall is costing the business.
- The filing cabinet (DuckDB) was found to silently stop seeing new updates under a very specific, easy-to-miss condition: if the dashboard held one long-running connection open instead of checking in fresh each time. This was caught by directly testing two real, separate processes against each other — one writing updates, one reading — and observing that the reader never saw the new data. The fix was to have the dashboard open a brand-new connection every single time it checks for updates, guaranteeing it never silently displays outdated numbers.

- The message-passing library connecting the different services also turned out to have two real defects of its own — one that silently failed to package messages correctly, and one that reported connection failures using the wrong error type. Both were caught by testing actual behavior rather than trusting the documentation, and both were patched with small, targeted fixes.